

Hierarchical Polyhedra Navigation for Macro Placement

A Framework for Decomposing NP-Hard Placement into Tractable Subproblems

1. Problem Reformulation

1.1 The Standard View

The macro placement problem minimizes a proxy cost function over continuous macro positions subject to canvas bounds, fixed-position constraints, and pairwise non-overlap constraints for hard macros. The non-overlap requirement for each pair (i, k) is a 4-term disjunction: at least one of $\{i \text{ left of } k, k \text{ left of } i, i \text{ below } k, k \text{ below } i\}$ must hold, but which one is not specified.

1.2 The Polyhedra View

We reformulate the problem as two coupled optimization problems by introducing an explicit **disjunctive assignment** σ .

Definition (Disjunctive Assignment). For n_h hard macros, a disjunctive assignment is a function:

$$\sigma : \{(i, k) : 1 \leq i < k \leq n_h\} \rightarrow \{L, R, A, B\}$$

where $L = "i \text{ left of } k"$, $R = "k \text{ left of } i"$ (equivalently, $i \text{ right of } k$), $A = "i \text{ above } k"$, $B = "i \text{ below } k"$. The domain has $|\{(i, k)\}| = n_h(n_h-1)/2$ elements.

Each assignment σ induces a set of linear inequality constraints. Combined with the canvas bounds, these define a convex polyhedron P_σ in R^{2n} (the space of all macro center coordinates). Some assignments σ produce empty polyhedra (infeasible constraint combinations). Let Σ_{feas} denote the set of feasible assignments.

Decomposition Theorem (restated). Given any $\sigma \in \Sigma_{\text{feas}}$, minimizing the HPWL wirelength over P_σ is a linear program, solvable in polynomial time. The full placement

problem is equivalent to:

$$\min_{\{\sigma \in \Sigma_{\text{feas}}\}} \left[\min_{\{p \in P_{\sigma}\}} f(p) \right]$$

where $f(p) = \text{WL}(p) + 0.5 \cdot D(p) + 0.5 \cdot C(p)$ is the proxy cost.

1.3 Two Decoupled Problems

Problem 1 (Navigation): Search over Σ_{feas} to find the assignment σ^* that minimizes the inner problem's optimal value. This is a combinatorial optimization over a discrete space. It is NP-hard.

Problem 2 (Evaluation): Given a fixed σ , find the optimal placement p^* within P_{σ} . The HPWL component is an LP. The density and congestion components introduce non-convexity but are treatable by short local refinement after the LP solve.

The key insight: Problem 2 is solved. LP solvers handle it in milliseconds. Problem 1 is where all difficulty resides, and it is the focus of this framework.

2. The Search Space

2.1 Raw Size

The number of possible disjunctive assignments is:

$$|\Sigma| = 4^{\{n_h(n_h-1)/2\}}$$

Benchmark	n_h	Pairs	$ \Sigma $	$\log_{10}(\Sigma)$
ibm01	246	30,135	$4^{\{30,135\}}$	$\sim 18,127$
ibm09	369	67,896	$4^{\{67,896\}}$	$\sim 40,847$
ibm18	537	143,916	$4^{\{143,916\}}$	$\sim 86,590$

This is intractable for enumeration or uniform random sampling.

2.2 Feasibility

The vast majority of assignments in Σ are infeasible. A simple example: if σ assigns "i left of j", "j left of k", and "k left of i", and the macros are too wide to all fit, the polyhedron P_{σ} is empty. The feasible subset Σ_{feas} is exponentially large but a vanishing fraction of Σ .

2.3 Navigability

Two assignments σ, σ' are **adjacent** if they differ in exactly one pair's label. The adjacency

graph on Σ_{feas} has:

- Vertices: all feasible assignments
- Edges: pairs differing in one label
- Degree: at most $3 \times n_h(n_h-1)/2$ (each pair can flip to 3 alternatives)

For ibm01, the neighborhood of any σ has at most 90,405 neighbors. The key empirical property (unproven but observed): most single-flip neighbors of a feasible assignment are themselves feasible when the current assignment comes from a legal placement with reasonable area utilization.

3. Hierarchical Decomposition

3.1 Motivation

Direct search over Σ_{feas} is intractable. We introduce a hierarchy that reduces the effective dimensionality of the search by separating coarse structural decisions from fine local adjustments.

3.2 Macro Grouping

Partition the set of hard macros into k groups $G_1, \dots, G_k$. Three natural partitioning strategies exist:

Connectivity-based: Apply a graph partitioner (spectral clustering, METIS, or Fiduccia-Mattheyses) to the netlist hypergraph. Macros sharing many nets are placed in the same group. This ensures that intra-group wiring (which the fine solver handles) captures the densest connectivity.

Size-based: Group macros by area quantile. Large macros dominate feasibility (they cause the most overlaps and most constrain the geometry), so resolving their relative positions first is most impactful.

Recursive bisection: Recursively partition the canvas and assign macros to spatial regions. At each level, solve a balanced min-cut problem: which macros go to which side to minimize net crossings? This produces a binary tree of spatial decisions.

The choice of k controls the coarse/fine trade-off:

k (groups)	Coarse pairs	Coarse search space	Fine pairs per group
5	10	$4^{\{10\}} \approx 10^6$	$\sim 2,400$ (avg)

10	45	$4^{\{45\}} \approx 10^{\{27\}}$	~300 (avg)
15	105	$4^{\{105\}} \approx 10^{\{63\}}$	~130 (avg)
20	190	$4^{\{190\}} \approx 10^{\{114\}}$	~75 (avg)

Numbers assume $n_h = 246$ (ibm01) with uniform group sizes.

3.3 Coarse Assignment

Definition (Coarse Assignment). A coarse assignment is a function:

$$\tau : \{(a,b) : 1 \leq a < b \leq k\} \rightarrow \{L, R, A, B\}$$

assigning a collective relative position to each pair of groups. $\tau(a,b) = L$ means "group G_a is to the left of group G_b ," which implies that every macro $i \in G_a$ is to the left of every macro $j \in G_b$.

A coarse assignment τ **pins** all inter-group pairwise labels simultaneously. For groups of size m , each inter-group relation pins m^2 pairwise assignments. The remaining **intra-group** pairwise assignments are unconstrained by τ .

3.4 The Cluster of Polyhedra

Definition (Cluster). Given a coarse assignment τ , the associated cluster is:

$$C(\tau) = \{ \sigma \in \Sigma_{\text{feas}} : \sigma \text{ is consistent with } \tau \}$$

This is the set of all fine-grained feasible assignments whose inter-group pairs match τ . Each $\sigma \in C(\tau)$ defines a polyhedron P_σ . The cluster $C(\tau)$ therefore corresponds to a family of polyhedra sharing the same coarse topology.

Cluster cost:

$$F(\tau) = \min_{\{\sigma \in C(\tau)\}} [\min_{\{p \in P_\sigma\}} f(p)]$$

This is the best achievable proxy cost among all placements consistent with coarse assignment τ .

3.5 The Quotient Space

Definition (Quotient Space). Define the equivalence relation $\sigma_1 \sim \sigma_2$ iff σ_1 and σ_2 agree on all inter-group pairs. The quotient space is:

$$\Sigma_{\text{feas}} / \sim = \{ C(\tau) : \tau \text{ is a feasible coarse assignment} \}$$

Each point in the quotient space is a cluster of polyhedra. The original optimization over Σ_{feas} factors as:

$$\min_{\{\sigma \in \Sigma_{\text{feas}}\}} f^*(\sigma) = \min_{\{\tau\}} \min_{\{\sigma \in C(\tau)\}} f^*(\sigma) = \min_{\{\tau\}} F(\tau)$$

where $f^*(\sigma) = \min_{\{p \in P_{\sigma}\}} f(p)$ is the LP-optimal cost for assignment σ .

4. Three-Level Architecture

4.1 Overview

The framework decomposes the problem into three nested optimization levels:

Level 1 (Coarse Topology):	$\min_{\{\tau\}} F(\tau)$	
Level 2 (Fine Assignment):	$\min_{\{\sigma \in C(\tau)\}} f^*(\sigma)$	[given τ from Level 1]
Level 3 (Continuous Solve):	$\min_{\{p \in P_{\sigma}\}} f(p)$	[given σ from Level 2]

Each level solves a smaller, more tractable problem than the one above it.

4.2 Level 3: Continuous Solver

Input: A complete disjunctive assignment σ . **Output:** Optimal macro positions $p^* \in P_{\sigma}$ and proxy cost $f(p^*)$.

Method: The HPWL objective is reformulated as an LP by introducing bounding-box variables for each net:

For each net S_j , introduce $u_j^{\{\max\}}, u_j^{\{\min\}}, v_j^{\{\max\}}, v_j^{\{\min\}}$

Minimize $\sum_j (u_j^{\{\max\}} - u_j^{\{\min\}} + v_j^{\{\max\}} - v_j^{\{\min\}})$

Subject to:

$$\begin{aligned} u_j^{\{\max\}} &\geq x_i && \text{for all } i \in S_j, \text{ for all } j \\ u_j^{\{\min\}} &\leq x_i && \text{for all } i \in S_j, \text{ for all } j \\ v_j^{\{\max\}} &\geq y_i && \text{for all } i \in S_j, \text{ for all } j \\ v_j^{\{\min\}} &\leq y_i && \text{for all } i \in S_j, \text{ for all } j \\ &&& [\text{half-space constraints from } \sigma] \\ &&& [\text{canvas bounds}] \end{aligned}$$

Size: For ibm01 (246 macros, 7269 nets): approximately 30,000 variables and 90,000

constraints. Solvable in ~1-10ms by modern LP solvers (HiGHS, CPLEX, Gurobi).

Warm-starting: When σ' differs from σ by a single pair flip, only 1-2 constraints change. The dual simplex method re-optimizes from the previous basis in $O(n)$ pivots rather than solving from scratch. This reduces per-evaluation cost to sub-millisecond.

Density and congestion refinement: After the LP solve, apply a short gradient-based refinement (~100 iterations) using the smooth log-sum-exp HPWL approximation plus density penalty, constrained to P_σ . This optimizes the full proxy cost, not just HPWL. Alternatively, evaluate the TILOS evaluator directly on the LP solution to get the true proxy cost.

4.3 Level 2: Fine Assignment Solver

Input: A coarse assignment τ (from Level 1) that fixes all inter-group pairwise relations.

Output: The best fine-grained assignment $\sigma^* \in C(\tau)$ found within a time budget.

Method: Local search with dual-variable-guided proposals.

1. **Initialize** σ_0 by extending τ : for each intra-group pair, assign the relation implied by the macro positions from a quick greedy placement consistent with τ .
2. **Evaluate** σ_0 via Level 3. Record the LP dual variables $d_{\{ik\}}$ for each pairwise constraint.
3. **Propose** σ' by flipping the intra-group pair (i,k) with the largest $|d_{\{ik\}}|$ (most expensive active constraint). Flip to the alternative that the dual suggests would improve cost most.
4. **Evaluate** σ' via Level 3 (warm-started LP).
5. **Accept/reject** using Metropolis criterion: accept if $f^*(\sigma') < f^*(\sigma)$, or with probability $\exp(-\beta(f^*(\sigma') - f^*(\sigma)))$ otherwise.
6. **Repeat** steps 3-5 for a fixed iteration budget.

Degrees of freedom: Only intra-group pairs are free. For $k=15$ groups with ~16 macros per group, each group has ~120 intra-group pairs. Total free variables: ~1,800 (vs. 30,135 in the full problem). This is a 17x reduction.

Cluster moves: Periodically, instead of flipping one pair, re-solve all intra-group assignments for a single group from scratch. Move all macros in G_a simultaneously by running a small-scale SA or greedy packer on just that group, holding all other groups fixed. This provides non-local jumps within the cluster $C(\tau)$.

4.4 Level 1: Coarse Topology Search

Input: The problem instance (macros, nets, canvas). **Output:** The best coarse assignment

τ^* found within a time budget.

Method: Stochastic search over the coarse space $\{L, R, A, B\}^{k(k-1)/2}$.

Evaluation: Each candidate τ is evaluated by running Level 2 (with a limited iteration budget), which returns $\hat{F}(\tau) \approx F(\tau)$. This is expensive (each evaluation involves many LP solves), so the number of τ evaluations is limited.

Search strategies (in order of increasing sophistication):

1. **Multi-start greedy:** Generate N random feasible coarse assignments. For each, run Level 2 briefly. Keep the best.
2. **Simulated annealing on τ -space:** Starting from the coarse assignment implied by the RePIAce baseline, propose single-group-pair flips, evaluate via Level 2, accept/reject with temperature schedule.
3. **Parallel tempering:** Run M chains at different temperatures on the τ -space. Hot chains explore broadly; cold chains exploit. Swap configurations between adjacent temperatures periodically. This addresses the disconnectedness problem at the coarse level.
4. **Bayesian optimization:** Treat $F(\tau)$ as a black-box function. Build a Gaussian process surrogate over the τ -space (using Hamming distance as the kernel metric). Use the surrogate to select promising τ candidates for full evaluation. This is appropriate when each evaluation is expensive but the space is moderate-dimensional.

Warm-starting from baselines: Extract the coarse assignment τ_{baseline} from the RePIAce solution. Begin the search from τ_{baseline} . The baseline is known to be good, so the search focuses on local improvements to the coarse topology rather than random exploration.

5. Initialization from Existing Solutions

5.1 Extracting σ from a Placement

Given a legal placement p (e.g., the RePIAce baseline output), the implied disjunctive assignment σ_p is:

```
For each pair  $(i, k)$  with  $i < k$ :  
    if  $x_i + w_i/2 \leq x_k - w_k/2$ :       $\sigma_p(i, k) = L$   
    elif  $x_k + w_k/2 \leq x_i - w_i/2$ :     $\sigma_p(i, k) = R$   
    elif  $y_i + h_i/2 \leq y_k - h_k/2$ :     $\sigma_p(i, k) = B$   
    elif  $y_k + h_k/2 \leq y_i - h_i/2$ :     $\sigma_p(i, k) = A$ 
```

```
else: ERROR (macros overlap)
```

Note: for non-overlapping macros, at least one condition holds. When multiple conditions hold (macros separated in both x and y), any valid choice works. The tie-breaking rule can affect the neighborhood structure of subsequent search.

5.2 Extracting τ from σ

Given a grouping $G_1, \dots, G_k$ and a fine assignment σ , the implied coarse assignment is:

```
For each group pair (a,b) with  $a < b$ :  
   $\tau(a,b)$  = majority vote of {  $\sigma(i,j)$  :  $i \in G_a, j \in G_b$  }
```

When the majority is not decisive (mixed L/R/A/B within a group pair), this signals that the grouping may be poorly chosen or that the coarse abstraction does not cleanly separate for this pair. In such cases, the coarse assignment for that pair can be left free for Level 2 to resolve.

5.3 Evaluating the Gap

After extracting σ_{baseline} from the RePIAce output, solve the LP within $P_{\{\sigma_{\text{baseline}}\}}$ to get the HPWL-optimal placement for that polyhedron. Compare this to the RePIAce placement's HPWL. The difference reveals how much room for improvement exists within the current polyhedron (the continuous slack) vs. requiring a different polyhedron (the combinatorial slack).

If the LP-optimal HPWL within $P_{\{\sigma_{\text{baseline}}\}}$ is close to RePIAce's HPWL, then the bottleneck is the polyhedron choice (Level 1/2). If there is a large gap, then even within the current polyhedron, better continuous optimization would help (Level 3).

6. Dual Variables as Gradient Information

6.1 Shadow Prices

The LP dual variable $d_{\{ik\}}$ associated with the pairwise constraint for pair (i,k) in assignment σ gives the **marginal cost of that constraint**: how much the optimal HPWL would decrease if the constraint were relaxed by one unit. A large $|d_{\{ik\}}|$ means the separation requirement between macros i and k is expensive – the placement would improve significantly if they could be closer together (or rearranged).

6.2 Proposal Distribution

Use dual variables to construct an informed proposal distribution for Level 2:

$$P(\text{flip pair } (i,k)) \propto |d_{ik}|^\alpha$$

where $\alpha > 0$ controls the sharpness. At $\alpha = 0$, proposals are uniform (random flips). At $\alpha \rightarrow \infty$, only the most expensive pair is ever proposed. Intermediate values ($\alpha \approx 1-2$) balance exploration and exploitation.

6.3 Direction of Flip

When flipping pair (i,k) from its current assignment (say L), the LP dual also suggests which alternative is most promising. Specifically:

- The current constraint (L): $x_i + w_i/2 \leq x_k - w_k/2$
- The dual $d_{ik} > 0$ means the constraint is binding and costly

Flipping to R ($x_k + w_k/2 \leq x_i - w_i/2$) reverses the horizontal ordering. Flipping to A or B changes to vertical separation. The expected improvement for each alternative can be estimated from the current LP solution without solving a new LP:

$$\Delta E(L \rightarrow R) \approx d_{ik} \cdot (\text{current separation in } x)$$

$$\Delta E(L \rightarrow A) \approx d_{ik} \cdot (\text{current separation in } x) - [\text{estimated y-constraint cost}]$$

These estimates are heuristic but cheap, and they focus the search on the most promising moves.

7. Computational Budget Allocation

7.1 Runtime Constraint

The competition enforces a 1-hour hard timeout per benchmark on AMD EPYC 9655P (16 cores) + RTX 6000 Ada 48GB.

7.2 Budget Model

Assume the following per-operation costs (to be calibrated empirically):

Operation	Time estimate	Notes
LP solve (cold start)	5-50 ms	Depends on n_h
LP solve (warm start)	0.1-1 ms	Single constraint change

TILOS proxy cost evaluation	10-50 ms	Full WL + Density + Cong
Level 2 iteration	1-5 ms	Warm LP + accept/reject
Level 2 full run (per τ)	1-10 s	1,000-5,000 iterations
Level 1 evaluation	1-10 s	One Level 2 run

7.3 Allocation Strategy

With a 3,600-second budget:

Phase 0 (Initialization):	60s	Extract baseline, set up LP, calibrate
Phase 1 (Coarse search):	1,200s	~100-200 coarse evaluations
Phase 2 (Fine refinement):	1,800s	Deep Level 2 run on best τ^*
Phase 3 (Final polish):	540s	Gradient-based continuous refinement

Parallelism: Level 1 evaluations are embarrassingly parallel across 16 cores. This gives ~1,600-3,200 coarse evaluations in Phase 1, dramatically expanding the search.

8. Surrogate Acceleration

8.1 Motivation

The bottleneck in Level 1 is the cost of evaluating $F(\tau)$ – each evaluation requires running Level 2, which involves many LP solves. A learned surrogate model $\hat{F}(\tau)$ that approximates $F(\tau)$ cheaply enables vastly more exploration.

8.2 Architecture

Input representation: Encode the coarse assignment τ as a vector in $\{0,1,2,3\}^{k(k-1)/2}$ (one-hot or ordinal encoding of the 4 labels). Augment with features derived from the grouping: total area per group, inter-group net count, group centroid distances.

Model: A small MLP or gradient-boosted tree that maps the encoded τ to a scalar cost prediction $\hat{F}(\tau)$.

Training data: Collect $(\tau, F(\tau))$ pairs from the Level 1 search. After 50-100 true evaluations, the surrogate begins to provide useful guidance. Retrain periodically as more data arrives.

Usage: In Level 1, use $\hat{F}(\tau)$ to pre-screen candidates. Evaluate the top-ranked candidates with the true evaluator. This is a standard Bayesian optimization / surrogate-assisted search

pattern.

8.3 Fine-Level Surrogate

A separate surrogate can accelerate Level 2. Train a model that predicts $f^*(\sigma)$ from σ (or from the change $\Delta\sigma$ relative to the current assignment). Use it for MCMC accept/reject decisions, validating with the true LP only for accepted moves or periodically.

9. Theoretical Properties

9.1 Completeness

The framework searches over the same feasible set as the original problem. Any legal placement corresponds to some $\sigma \in \Sigma_{\text{feas}}$ and some $p \in P_{\sigma}$. The hierarchical decomposition introduces no approximation in the problem formulation – only in the search strategy.

9.2 Refinement Guarantee

Starting from the baseline assignment σ_{baseline} , the Level 2 local search is guaranteed to find a solution at least as good as the baseline (by accepting only improving moves or using the baseline as the initial point of the Metropolis chain). The hierarchical search can only improve upon this.

9.3 Benders Decomposition Connection

The three-level architecture is a Benders decomposition:

- **Master problem:** Level 1 (coarse assignment τ)
- **Subproblem:** Levels 2+3 (fine assignment and continuous solve, given τ)
- **Benders cuts:** The dual information from the subproblem (which inter-group constraints are most expensive) provides valid inequalities that can be added to the master problem to prune the search.

In classical Benders decomposition, the cuts provide a provably converging lower bound. Here, the non-convexity of density/congestion components prevents exact Benders cuts, but the dual-based guidance still provides directional information.

9.4 Relationship to Standard Pipeline

The standard relaxation-legalization-refinement pipeline can be viewed as a degenerate case of this framework:

- Level 1 is skipped entirely (no coarse topology optimization)
- Level 2 is performed once, greedily, irreversibly (legalization)
- Level 3 is local search within the single resulting polyhedron

The key difference is reversibility. In the standard pipeline, the legalization step commits to a polyhedron permanently. In this framework, the polyhedron choice is revisable at both Level 1 and Level 2, allowing exploration of fundamentally different non-overlapping arrangements.

10. Implementation Roadmap

Phase 1: Proof of Concept (ibm01 only)

1. Parse ibm01 benchmark into macro dimensions, net connectivity, canvas size
2. Extract σ_{baseline} from the RePIAce baseline placement
3. Formulate and solve the HPWL LP within $P_{\{\sigma_{\text{baseline}}\}}$ using HiGHS
4. Compare LP-optimal HPWL to RePIAce HPWL (quantify continuous slack)
5. Implement single-pair-flip proposal with dual variable ranking
6. Run greedy local search (accept only improvements) for 10,000 iterations
7. Report: number of improving polyhedra found, total cost reduction

Phase 2: Full Level 2 with MCMC

1. Implement Metropolis-Hastings with temperature parameter
2. Implement LP warm-starting for single-constraint changes
3. Implement cluster moves (re-solve one group from scratch)
4. Calibrate temperature schedule and proposal distribution on ibm01-ibm04
5. Benchmark: cost vs. iteration count, wall-clock time per iteration

Phase 3: Hierarchical (Levels 1+2+3)

1. Implement connectivity-based grouping via spectral clustering on netlist
2. Implement coarse assignment extraction from baseline

3. Implement Level 1 search (SA on τ -space with Level 2 as evaluator)
4. Parallelize Level 1 evaluations across 16 cores
5. Benchmark on all 17 IBM instances

Phase 4: Acceleration and Polish

1. Train surrogate models for Level 1 and Level 2
 2. Implement multi-fidelity evaluation cascade
 3. Implement parallel tempering at Level 1
 4. Full proxy cost evaluation (WL + Density + Congestion) using TILOS evaluator
 5. Final gradient-based continuous refinement within best polyhedron
 6. Optimize budget allocation across phases
-

11. Open Questions

1. **Optimal group count k .** What is the right coarse/fine trade-off for the ICCAD04 benchmarks? This requires empirical calibration.
2. **Feasibility of neighbors.** What fraction of single-flip neighbors of a feasible assignment are themselves feasible? If this fraction is low, the local search is severely constrained. Empirical measurement on extracted baseline assignments will answer this.
3. **Continuous vs. combinatorial slack.** How much of the gap between the RePIAcE baseline and the global optimum is due to suboptimal continuous positions (fixable within the current polyhedron) vs. suboptimal topology (requiring a different polyhedron)? The Phase 1 proof-of-concept will answer this.
4. **Density and congestion under LP placement.** The LP optimizes only HPWL. Does the LP-optimal placement within a given polyhedron produce reasonable density and congestion, or does it create pathological clustering? If the latter, the continuous refinement step (Level 3) must be more aggressive.
5. **Scaling to ibm18.** The LP size grows quadratically with n_h . At 537 macros, the LP has $\sim 290,000$ variables and $\sim 870,000$ constraints. Is warm-started dual simplex still sub-millisecond at this scale?
6. **Integration with advanced navigation.** The framework is designed to be modular at Level 1. The coarse search can be replaced by any method that searches over

$\{L, R, A, B\}^{k(k-1)/2}$: MCMC, parallel tempering, Bayesian optimization, discrete Schrodinger bridges, survey propagation, or AlphaEvolve-style LLM-guided evolution. The architecture does not commit to any single method.